Manuel Utilisateur

Compilateur Deca - Projet Génie Logiciel

Équipe GL25

Nathan Nicolas Alexandre Iles Noah

Janvier 2026

Résumé

Ce document constitue le manuel utilisateur du compilateur Deca développé par l'équipe GL25. Il décrit les options de la commande `decac`, les messages d'erreur possibles, les limitations de l'implémentation et les extensions disponibles.

Table des matières

1 Le compilateur **3**
1.1 Ses fonctionnalités . . . 3
1.2 Ses limitations . . . 3

2 Utilisation de la commande decac **4**
2.1 Utilisation du compilateur sur Ensipc . . . 4
2.2 Syntaxe générale . . . 4
2.3 Raccourcis . . . 4
2.4 Options disponibles . . . 5
2.5 Exemples d'utilisation . . . 5

3 Messages d'erreur **6**
3.1 Erreurs de ligne de commande . . . 6
3.2 Erreurs lexicales . . . 7
3.3 Erreurs syntaxiques . . . 7
3.4 Erreurs contextuelles . . . 7
3.4.1 Variables et paramètres . . . 7
3.4.2 Classes, champs et méthodes . . . 7
3.4.3 Identificateurs et types . . . 8
3.4.4 Opérateurs . . . 8
3.4.5 Accès aux membres et appels de méthodes . . . 8
3.4.6 Autres erreurs contextuelles . . . 9
3.5 Erreurs à l'exécution . . . 9

4 Optimisations **10**
4.1 Niveau 1 : Simplifications locales . . . 10
4.2 Niveau 2 : Calculs anticipés . . . 10
4.3 Niveau 3 : Registres dédiés . . . 10
4.4 Mode sans vérifications . . . 11
4.5 Évaluation expérimentale des optimisations . . . 12
4.6 Résumé des niveaux . . . 12

5 Extension RISC-V **12**
5.1 Description . . . 12
5.2 Architecture . . . 12
5.3 Utilisation . . . 13
5.4 Fonctionnalités implémentées . . . 13
5.5 Limitations . . . 13
5.6 Outils externes . . . 14

1 Le compilateur

1.1 Ses fonctionnalités

Le compilateur `decac` permet de compiler des programmes écrits en langage Deca vers du code assembleur IMA, exécutable par la machine virtuelle `ima`.

Notre implémentation prend en charge :

— La partie **sans objet** du langage Deca (types primitifs, expressions, structures de contrôle)

— La partie **avec objet** du langage Deca (classes, héritage, méthodes, champs, cast)

— Des **optimisations** à plusieurs niveaux (voir section 4)

— Une **Représentation Intermédiaire (IR)** Three Address Code (TAC) fonctionnelle et testée pour toute la partie sans objet.

— Plusieurs backend de démonstration (IMA, RISC-V, QBE) compatibles avec notre IR.

1.2 Ses limitations

— **Taille de la pile :** La pile IMA est limitée. Une récursion trop profonde provoque un stack overflow.

— **Taille du tas :** Le tas pour les objets est limité. Trop d'allocations provoquent un heap overflow.

— **Nombre de registres :** 16 registres maximum (R0-R15), dont certains réservés.

2 Utilisation de la commande decac

2.1 Utilisation du compilateur sur Ensipc

Pour utiliser le compilateur sur une machine ensipc sans avoir de problème, il faut utiliser la commande suivante :

```
source ./docker/bashrcENSIPC
```

Un point important est qu'il n'est pas possible d'utiliser les backends QBE et RISC-V sur une machine de l'Ensimag au vu des installations que cela nécessite (RARS, QBE). Cependant, la Représentation Intermédiaire implémentée est affichable et branchable sur IMA.

2.2 Syntaxe générale

```
decac [options] <fichier.deca> ...
```

Le compilateur `decac` prend en entrée un ou plusieurs fichiers source Deca (extension `.deca`) et produit des fichiers assembleur IMA (extension `.ass`).

2.3 Raccourcis

Les raccourcis suivants sont disponibles après avoir chargé l'environnement Docker fourni avec le projet :

```
source ./docker/bashrc
```

Principe. Les commandes définies permettent d'invoquer directement les outils du projet (analyse lexicale, syntaxique, contextuelle, compilation et exécution), en retrouvant automatiquement les fichiers dans l'arborescence de tests.

Raccourcis principaux.

- `lex <fichier>` : analyse lexicale
- `parse <fichier>` : analyse syntaxique
- `sema <fichier>` : analyse contextuelle
- `decac [options] <fichier>` : compilation Deca
- `run <fichier.ass>` : exécution du code assembleur IMA
- `perf <fichier.ass>` : exécution en mode performance

Exemple minimal.

```
decac ln2.deca
run ln2.ass
```

2.4 Options disponibles

Option	Description
`-b`	Affiche la bannière (nom de l'équipe). Ne peut pas être combinée avec d'autres options.
`-p`	Arrête la compilation après l'étape de parsing et affiche l'arbre syntaxique (decompilation). Incompatible avec `-v`.
`-v`	Arrête la compilation après l'étape de vérification contextuelle. Incompatible avec `-p`.
`-n`	Désactive les vérifications à l'exécution (débordements, déréférencement null, etc.). Permet un code plus rapide mais moins sûr.
`-r X`	Limite le nombre de registres utilisables à X (entre 4 et 16 inclus). Utile pour tester la gestion des registres.
`-d`	Active le mode debug. Peut être répété pour augmenter le niveau de verbosité (`-d` = INFO, `-d -d` = DEBUG, `-d -d -d` = TRACE).
`-P`	Active la compilation parallèle. Compile plusieurs fichiers simultanément pour améliorer les performances.
`-w`	Active l'affichage des warnings (fichiers dupliqués, etc.).
`-o`	Active les optimisations. Peut être répété pour augmenter le niveau d'optimisation (voir section 4).

2.5 Exemples d'utilisation

```
# Compilation simple
decac programme.deca

# Afficher la banniere
decac -b

# Parsing uniquement (affiche l'arbre)
decac -p programme.deca

# Verification contextuelle uniquement
decac -v programme.deca

# Compilation avec optimisations maximales
decac -o -o -o programme.deca

# Compilation parallele de plusieurs fichiers
decac -P fichier1.deca fichier2.deca fichier3.deca

# Compilation avec registres limites et debug
decac -r 4 -d -d programme.deca
```

3 Messages d'erreur

Cette section liste les messages d'erreur retournés par le compilateur. Chaque message est préfixé par la localisation dans le fichier source sous la forme `<nom de fichier.deca>:<ligne>:<colonne>: <description informelle du problème>`.

3.1 Erreurs de ligne de commande

Ces erreurs sont préfixées par `Error during option parsing:`.

Message	**Configuration**
`Error: Option -b cannot be combined with any other option.`	`decac -b -p file.deca`
`Error: Options -v and -p cannot be combined.`	`decac -p -v file.deca`
`Error: Option -r must be followed by an integer between 4 and 16 (inclusive).`	`decac -r` (sans argument)
`Error: The number of registers must be between 4 and 16 (inclusive).`	`decac -r 3 file.deca`
`Error: Option -w can only be specified once.`	`decac -w -w file.deca`
`Error: Option -n can only be specified once.`	`decac -n -n file.deca`
`Error: Option -P can only be specified once.`	`decac -P -P file.deca file2.deca`
`Error: File <path> does not exist at the specified location.`	`decac inexistant.deca`
`Error: <path> is not a valid filename.`	Chemin vers un répertoire
`Error: File <path> is unreadable (check permissions).`	Fichier sans droits de lecture
`Error: File <path> has wrong format (.deca extension expected).`	`decac fichier.txt`
`Error: At least one file is required for this option`	`decac -p` (sans fichier)

3.2 Erreurs lexicales

Message	Configuration
`Error during lexical analysis (integer) => [Overflow for input: "<val>"]`	`int x = 2147483648;`
`Error during lexical analysis (float) => [Overflow for input: "<val>"]`	`float x = 3.4028235e+39;`
`Error during lexical analysis (float) => [Underflow for input: "<val>"]`	`float x = 1.0e-241;`

3.3 Erreurs syntaxiques

Message	Configuration
`<fichier>: include file not found`	`#include "inexistant.decah"`
`Circular include for file <fichier>`	Inclusion circulaire A $\rightarrow$ B $\rightarrow$ A
`left-hand side of assignment is not an lvalue`	`1 = a;`

3.4 Erreurs contextuelles

3.4.1 Variables et paramètres

Message	Configuration
`Forbidden declaration (void)`	`{ void x; }`
`Variable already declared: <nom>`	`{ int x; int x; }`
`Forbidden parameter type (void)`	`void m(void param) {}`
`Parameter already declared: <nom>`	`void m(int a, int a) {}`

3.4.2 Classes, champs et méthodes

Message	Configuration
`Class <nom> already declared`	`class C {} class C {}`
`Unrecognised superClass <nom> during declaration`	`class Fille extends Mere {}`
`Class <nom> is not of type 'class'`	`class B extends int {}`
`Field <nom> cannot be of type void`	`class A { void champ; }`
`Field <nom> already defined`	`class A { int x; float x; }`
`The Overriden Field <nom> was not a field`	Redéfinition d'une méthode comme champ
`Method <nom> already defined`	`class A { int m(){} float m(){} }`

Message	Configuration
`The Overriden Method <nom> was not a method`	Redéfinition d'un champ comme méthode
`Override Method <nom> must have the same signature...`	Signature différente lors de la redéfinition
`Override Method <nom> must return a subtype...`	Type de retour non covariant

3.4.3 Identificateurs et types

Message	Configuration
`Identifier '<nom>' not declared`	`{ x = 3; }`
`Type <type> incompatible with expected type <attendu>...`	`{ int x; x = true; }`
`Condition must be a boolean. Got: <type>`	`{ if (2) {} }`

3.4.4 Opérateurs

Message	Configuration
`Arithmetic operation operands must be numeric`	`{ float f; f = 1 - true; }`
`Modulo Operator Only Takes Integers`	`{ int x = 5.0 % 2; }`
`Boolean operator can only be applied to boolean values`	`{ boolean b = 5 && true; }`
`Inequality operator can only be applied to numeric values`	`{ boolean b = true < false; }`
`Unary Not Only Takes Boolean`	`{ boolean b = !5; }`

3.4.5 Accès aux membres et appels de méthodes

Message	Configuration
`Fields Selections must be done on Classes`	`{ int x = 5; int y = x.champ; }`
`This Class is not declared`	Classe non trouvée dans l'environnement
`Fields Selections must be Fields`	Accès à une méthode comme un champ
`This Field is PROTECTED thus can't be reached`	`Mere m = new Mere();` `println(m.var);`
`Method call must be on a class`	`{ int x = 5; x.methode(); }`
`Identifier '<nom>' not declared`	`obj.methodeInexistante();`

Message	Configuration
`Identifier '<nom>' cannot be used as Method`	`obj.champ();`
`Wrong number of parameters`	`obj.methode(1);` (attend 2 params)

3.4.6 Autres erreurs contextuelles

Message	Configuration
`print: type <type> not allowed`	`{ boolean x; print(x); }`
`Cannot return a value from a void method`	`void m() { return 1; }`
`Invalid instanceOf expression`	`{ if (623825122 instanceof int){} }`
`Incompatible types for cast`	`Voiture v = (Voiture) animal;`
`This can not be called in null class`	`{ MaClasse obj = this; }`
`'new' must be followed by a class identifier`	`{ int x = new int(); }`

3.5 Erreurs à l'exécution

Ces erreurs surviennent lors de l'exécution du code assembleur généré (sauf si `-n` est utilisé).

Message	Configuration
`Error: Overflow during arithmetic operation`	`3.4e+38 * 1.0e+38`
`Error: Division by zero`	`5 / 0`
`Error: Null dereference`	Accès sur `null`
`Error: Stack overflow`	Récursion trop profonde
`Error: Heap overflow`	Trop d'allocations
`Error: Input/Output error`	Entrée invalide pour `readInt()`/`readFloat()`

4 Optimisations

Le compilateur propose trois niveaux d'optimisation cumulatifs, activés par l'option `-o`.

4.1 Niveau 1 : Simplifications locales

```
decac -o fichier.deca
```

Ce niveau active des optimisations locales (à l'échelle d'une expression ou d'un petit voisinage d'instructions), sans modifier la structure de contrôle du programme.

Optimisations activées :

- **Redondances** : suppression de séquences LOAD/STORE inutiles quand une valeur est déjà disponible
- **Calcul en place** : calcul directement dans le registre cible plutôt que via un registre temporaire
- **Réordonnancement commutatif** : sur `+` et `*`, permutation contrôlée lorsque la valeur de droite est plus complexe (algorithme de Sethi-Ullman)
- **Cas spéciaux de comparaison** : spécialisations pour `x == 0`, `x != 0`, et comparaisons avec opérande déjà en registre
- **Réécritures ciblées** : ex. division float par `2.0` → multiplication par `0.5`

Exemple :

```
m = (g+d) / 2.0f;   // Division reecrite en multiplication par 0.5
```

4.2 Niveau 2 : Calculs anticipés

```
decac -o -o fichier.deca
```

Ce niveau introduit l'évaluation à la compilation dès qu'une expression est prouvée constante.

Optimisations activées :

- **Constant folding** : si les deux opérandes sont constants, l'expression est évaluée à la compilation
- **Propagation des constantes** : une sous-expression constante devient réutilisable

Exemple :

```
// Ces divisions sont calculees a la compilation
float a = 1.0/5040.0;  // -> 0.000198...
float b = 1.0/720.0;   // -> 0.001388...
float c = 1.0/120.0;   // -> 0.008333...
```

4.3 Niveau 3 : Registres dédiés

```
decac -o -o -o fichier.deca
```

Ce niveau active des optimisations plus agressives : allocation dédiée de registres pour certaines variables et transformations qui peuvent ajouter une garde runtime (branche) pour préserver la sémantique.

Optimisations activées :

— **Variables épinglées** : promotion de variables locales sélectionnées dans des registres réservés (moins d'accès mémoire, flush uniquement si modifiées).

— **Division par 2^k (int) avec garde de signe** : `x / 8` utilise des décalages sur le chemin non-négatif (optimisation Niveau 3, car résolue au *runtime*), sinon bascule sur la division standard.

— **Modulo 2^k (int) avec garde de signe** : `x % 4` est calculé via décalages + soustraction sur le chemin non-négatif (optimisation Niveau 3, car résolue au *runtime*), sinon fallback sur REM.

Exemple :

```
// u%2 et u/2 remplaces par des decalages (si u >= 0)
if (u%2 == 0) {
    u = u/2;    // -> u = u >> 1 (si u >= 0)
} else {
    ...
}
```

Pourquoi plus d'instructions mais plus rapide ?

Ces transformations reposent sur la création de *guards* au runtime. Le code généré teste d'abord si l'opérande est une valeur « interdite » par l'optimisation (ex : négative).

— Sur le **chemin chaud** (cas fréquent), l'opération complexe est remplacée par des instructions élémentaires (décalages), dont le coût en cycles CPU est négligeable par rapport à une division.

— Si la valeur est interdite, un mécanisme de *fallback* redirige l'exécution vers l'instruction standard (plus lente).

Le gain de performance sur le chemin optimisé compense largement le surcoût de la vérification initiale.

Hot path : Moins d'accès mémoire, opérations renforcées (shifts), avec fallback sûr si nécessaire.

4.4 Mode sans vérifications

```
decac -n fichier.deca
```

L'option `-n` désactive la génération des tests de sécurité à l'exécution (comportement indéfini en cas d'erreur), **à l'exception des vérifications d'Entrées/Sorties (I/O)** qui restent actives.

Concrètement, cette option entraîne :

— **Suppression** des vérifications de débordement arithmétique (BOV).

— **Suppression** des tests de division par zéro.

— **Suppression** des tests de déréférencement de pointeur null.

— **Maintien des erreurs de lecture (ex : valeur entrée invalide).**

Conséquence : Gain de performance maximal, mais comportement indéfini en cas d'erreur.

Attention : L'option `-n` est à réserver aux exécutions maîtrisées sur du code certifié correct. À proscrire en développement ou avec des entrées utilisateur.

4.5 Évaluation expérimentale des optimisations

Évaluation expérimentale : Les performances obtenues avec les différents niveaux d'optimisation ont été mesurées sur des programmes de référence. Les résultats détaillés et analyses associées sont accessibles à l'adresse suivante : `perf_gl25`

4.6 Résumé des niveaux

Niveau	Options	Description
1	`-o`	Simplifications locales : nettoyage, calcul en place, réécritures
2	`-o -o`	Calculs anticipés : constant folding, propagation des constantes
3	`-o -o -o`	Registres dédiés : variables épinglées, divisions/-modulos par 2^k
Max	`-o -o -o -n`	Performance maximale, vérifications désactivées

5 Extension RISC-V

5.1 Description

Notre extension implémente un backend multi-cibles via une représentation intermédiaire (IR). Nous avons volontairement choisi de travailler sur la modularité du compilateur pour axer nos efforts sur la création d'une IR fonctionnelle et robuste. Ainsi, le compilateur utilise un frontend commun (Lexer, Parser, Contexte) qui produit une IR générique, avant que trois backends différents ne génèrent le code final.

5.2 Architecture

1. **Frontend** : Analyse lexicale, syntaxique et contextuelle
2. **IR** : Représentation intermédiaire générique (IRProgram, IRInst, IRData, IRLabel)
3. **Backends** :
   — **IMA** : Génère du code `.ass` pour la machine abstraite IMA
   — **RISC-V** : Génère du code `.s` pour le simulateur RARS
   — **QBE** : Génère du code `.qbe` pour le backend universel QBE (Spécification QBE)

5.3 Utilisation

L'extension est disponible et fonctionnelle en utilisant certaines options décrites ci-dessous.

```
# Backend IMA (option -ir 1)
$ decac -ir 1 hello-world.deca
$ ir_ima hello-world.ass
Hello, world!

# Backend RISC-V (option -ir 2)
$ decac -ir 2 hello-world.deca
$ ir_rars hello-world.s
Hello, world!

# Backend QBE (option -ir 3)
$ decac -ir 3 hello-world.deca
$ ir_qbe hello-world.qbe
```

5.4 Fonctionnalités implémentées

Les fonctionnalités suivantes sont supportées par l'IR (génération du code intermédiaire) :

- **IR générique** : IRProgram, IRAbstractInst, IRData, IRLabel
- **Trois backends** : IMA, RISC-V (RARS), QBE (partiels)
- **Entrées/sorties** : `print`, `println`, `exit`
- **Arithmétique** : ADD, SUB, MUL, DIV, MOD (INT et FLOAT)
- **Variables** : déclaration, chargement, affectation (stack/heap)
- **Structures de contrôle** : `if`/`else`, `while`
- **Comparaisons** : >, >=, <, <=, ==, != avec branchements
- **Booléens** : AND, OR avec évaluation en court-circuit
- **Conversions** : int vers float (IRConvFloat)
- **Environnement Docker** : RARS et QBE intégrés

5.5 Limitations

Les fonctionnalités suivantes ne sont pas implémentées dans l'IR :

- **Backends incomplets** : seuls les prints fonctionnent (IMA/QBE), .data uniquement (RISC-V)
- Allocation de registres
- Appels de méthodes/fonctions
- Programmation orientée objet (classes, héritage, dispatch)
- Tableaux et structures complexes

5.6 Outils externes

Outil	**Description**
`RARS`	RISC-V Assembler & Runtime Simulator
`QBE`	Compiler Backend universel (AMD64/ARM/RISC-V)